# Benchmark des modèles locaux (Ollama, CPU) — étape 3

*Piste : « 3 questions qui débloquent les briefs ». Mis à jour le 11/11/2026 : les 6 modèles prévus sont maintenant tous mesurés. Chiffres bruts :* `benchmark/runs/<pc>.json` *; tableaux complets régénérés par* `python benchmark/run_benchmark.py --rapport` *→ tableaux.md.*

## 1. Ce qu'on mesure, et pourquoi

| Tâche | Usage dans le produit | Cas de test | Mesures |
|---|---|---|---|
| **Recherche** : besoin du commercial en langage courant → bonne question de la banque (24 questions) | Le junior tape « je ne sais pas quand ils lancent » et retrouve la question qui marche | 15 besoins, avec la rubrique et la ou les questions attendues | Bonne rubrique au rang 1, bonne question au **rang 1**, dans le **top 3** (les 3 questions montrées au junior), dans le top 5, MRR, temps par requête |
| **Extraction** (LLM) : note du rendez-vous précédent → rubriques du brief déjà connues, en JSON | Savoir quelles rubriques sont encore vides, pour choisir les 3 questions (entrée `rubriques_connues` de `POST /questions`) | 15 notes, avec les rubriques attendues | JSON valide, précision, rappel, F1, notes avec rubrique inventée, notes avec valeur inventée, temps par note |

**Méthode :**

- Les 15 cas (briefs_test.csv) ont été écrits **avant** le premier prompt et n'ont pas été modifiés ensuite.
- Les besoins sont volontairement formulés avec d'autres mots que les questions de la banque, pour tester la compréhension du sens.
- Réglages : Ollama sur `127.0.0.1`, `temperature 0`, graine 42, au plus 400 tokens par réponse, vecteurs de la banque en cache.
- Le TF-IDF sert de référence sans IA.
- Tous les modèles ont été mesurés sur le même PC, dans le même passage (11/11/2026).

## 2. Contrainte matérielle : mon PC

| | **PC principal (le mien)** |
|---|---|
| Processeur | AMD Ryzen 5 PRO 4650U (6 cœurs / 12 threads), carte graphique intégrée → **CPU uniquement** |
| RAM | 15,3 Go au total, mais **seulement 0,6 à 2,4 Go libres** pendant les tests (navigateur, VS Code, API ouverts) |
| Disque C: (emplacement par défaut des modèles Ollama) | **0,3 Go libres le 10/11**, 1,6 Go le 11/11, sur 237 Go |
| Disque D: | ~119 Go libres → **c'est là que sont rangés les modèles du benchmark** |
| Système | Windows 11 Pro, Ollama 0.40.16, Python 3.11.9 |

**Conséquences :**

- **Disque :** le 10/11, seuls `nomic-embed-text` et `qwen2.5:1.5b` tenaient sur C:. Solution : une **deuxième instance d'Ollama** (port 11435) qui range ses modèles sur D: (`OLLAMA_MODELS=D:\ollama\models`). Les 6 modèles y ont été téléchargés, environ 4,8 Go au total, sans toucher à l'Ollama habituel.
- **Modèles abandonnés :** `mxbai-embed-large` et `llama3.2:3b` faute de temps, mais ils peuvent être ajoutés de la même façon.
- **RAM :** avec 2 Go libres environ, Windows déplace une partie de la mémoire sur le disque. Ça ralentit surtout `qwen2.5:3b` (jusqu'à 62 s pour une note) et rend les temps variables d'un passage à l'autre. Ce sont des ordres de grandeur.
- **Coéquipiers :** sur un PC avec plus de RAM libre, il suffit de relancer le même script (`--pc pc2`) pour comparer les temps. Les scores de qualité ne devraient pas changer (temperature 0).

## 3. Résultats — recherche de la bonne question (15 cas)

| **Modèle** | **Taille** | **Bonne rubrique au rang 1** | **Bonne question au rang 1** | **Bonne question dans le top 3** | **Bonne question dans le top 5** | **MRR** | **Temps / requête** | **Indexation de la banque** |
|---|---|---|---|---|---|---|---|---|
| TF-IDF (référence, sans IA) | 0 | 40 % | 33 % (5/15) | 47 % (7/15) | 53 % | 0,44 | **0,1 ms** | instantané |
| nomic-embed-text | 274 Mo | 53 % | 47 % (7/15) | 60 % (9/15) | 73 % | 0,56 | 100–130 ms | 2,4 s |
| all-minilm | 45 Mo | 73 % | 67 % (10/15) | 80 % (12/15) | 87 % | 0,76 | **26–40 ms** | 0,8 s |
| **bge-m3** | 1,2 Go | **93 %** | **93 % (14/15)** | **100 % (15/15)** | **100 %** | **0,96** | 150–300 ms | 10,4 s |

L'indexation n'a lieu qu'une fois : les vecteurs sont ensuite lus dans le cache.

**Lecture :**

- Sur 15 besoins, bge-m3 met la bonne question dans les 3 proposées **15 fois**, contre 12 pour all-minilm, 9 pour nomic-embed-text et 7 pour le TF-IDF.
- bge-m3 est un modèle **multilingue**, ce qui explique sans doute son avance sur des phrases en français.

## 4. Résultats — extraction JSON des rubriques connues (15 notes)

**Le garde-fou :**

- C'est une règle simple, sans IA, ajoutée **après** le premier passage : une rubrique ne compte que si sa valeur est non vide et si ses mots se retrouvent dans la note.
- Comme elle a été décidée en voyant ces résultats, elle **devra être confirmée sur les 30 cas de validation** de l'étape 4.

| Modèle | Taille | JSON valide | Précision | Rappel | F1 | F1 avec garde-fou | Notes exactes avec garde-fou | Notes avec rubrique inventée (brut → garde-fou) | Notes avec valeur inventée | Temps / note (moy. / max) | Chargement |
|---|---|---|---|---|---|---|---|---|---|---|---|
| qwen2.5:0.5b | 397 Mo | 73 % | 0,23 | 0,67 | 0,34 | 0,41 | 7 % (1/15) | 73 % → 47 % | 60 % | **6,7 s** / 12,5 s | 5 s |
| qwen2.5:1.5b | 986 Mo | **100 %** | 0,44 | 0,92 | 0,59 | 0,72 | 7 % (1/15) | 100 % → 87 % | 60 % | 8,3 s / 11,4 s | 12 s |
| **qwen2.5:3b** | 1,9 Go | 93 % | 0,39 | 0,78 | 0,52 | **0,81** | **47 % (7/15)** | 93 % → **20 %** | 93 % | 24,5 s / 62,1 s | 21 s |

**Lecture :**

- **Sans garde-fou, les trois modèles cochent presque toutes les rubriques.** Leur précision brute est faible (0,23 à 0,44).
- **Le garde-fou profite surtout au 3b** : ses valeurs sont plus proches du texte de la note, donc la règle filtre mieux ses inventions. Il passe à 47 % de notes exactes, contre 7 % pour les deux autres.
- **Le 0.5b est le moins fiable :**
  - plus d'un JSON sur quatre est invalide ;
  - il invente des codes de rubrique qui n'existent pas (« valideurs », « valores ») ;
  - lors d'un premier passage, il a bouclé sans fin (plus de 17 000 tokens pour une note). D'où le plafond de 400 tokens par réponse, ajouté pour tous les modèles.

## 5. Trois exemples concrets

**Exemple 1 : les embeddings comprennent le sens, le TF-IDF compare les mots (cas T14).**

- Besoin : « Quel public vise-t-on : les jeunes, les familles ou le haut de gamme ? »
- TF-IDF : il propose en n°1 Q02 (« création, rénovation ou extension de gamme ? »), une question hors sujet.
- Les trois modèles d'embeddings proposent en n°1 Q05 (« plutôt un public jeune, familial ou premium ? »), la bonne réponse, alors que « haut de gamme » et « premium » ne partagent aucun mot.

**Exemple 2 : un piège de vocabulaire que seul bge-m3 évite (cas T09).**

- Besoin : « Quand est-ce que ça doit arriver dans les magasins ? »
- La bonne réponse est Q19 (« à quelle date le produit doit-il être en rayon ? »).
- nomic-embed-text et all-minilm proposent Q17 (« à quel prix le produit fini sera-t-il vendu **en magasin** ? »). Le mot « magasin » les attire : Q19 arrive seulement au rang 15 pour nomic et au rang 10 pour all-minilm.
- TF-IDF : Q19 au rang 3, derrière la question générique Q21.
- **bge-m3 : Q19 au rang 1.** Il a compris que la question porte sur une **date**, pas sur un prix.

**Exemple 3 : les LLM inventent, le garde-fou limite les dégâts (cas T03).**

- Note : « Lancement prévu en septembre 2027 avec trois tours de validation. Ils n'ont pas encore d'idée précise sur le produit. » La seule rubrique attendue est `delai`.
- **Brut :** les trois LLM cochent 7 ou 8 rubriques. Le 1.5b invente `"cible": "femme"` et prend le nom du client (« Ondevia ») pour une référence. Le 0.5b invente une rubrique `valideurs`.
- **Avec le garde-fou :**
  - il reste `delai` + `references` pour le 1.5b ;
  - `delai` + `type_produit` pour le 3b ;
  - toujours 8 rubriques pour le 0.5b.
- **Aucun n'est exact sur ce cas.** La règle ne détecte pas une erreur de sens quand le mot est bien écrit dans la note.

## 6. Recommandation pour la V2

1. **Recherche : bge-m3, avec all-minilm puis TF-IDF en repli.**
   - bge-m3 est le meilleur sur toutes les mesures : 100 % de bonnes questions dans le top 3, et +40 points sur nomic-embed-text. Le critère fixé à l'avance (« au moins +10 points sur le top 3 ») est largement rempli.
   - Il répond en 0,15 à 0,3 s sur CPU, ce qui est acceptable.
   - Repli si bge-m3 manque ou si la machine est trop faible : all-minilm, très léger (45 Mo, environ 30 ms, 80 % au top 3).

    - Si Ollama ne répond pas : TF-IDF (aucune dépendance).
    - nomic-embed-text n'est plus retenu.
2. **Extraction : pré-remplissage, pas d'automatisme, avec qwen2.5:3b + garde-fou.**
    - Le critère fixé à l'avance n'est **pas atteint** : le 3b obtient un F1 de 0,81 avec garde-fou pour 0,85 visé, et 24,5 s par note pour moins de 20 s visées.
    - On ne remplit donc pas les rubriques automatiquement. Le 3b **propose** les rubriques connues et le junior les **confirme en un clic**, ce qui va dans le sens du produit.
    - Le temps n'est pas bloquant : la préparation se fait **avant** le rendez-vous, et le calcul peut tourner en arrière-plan.
    - Sur une machine sans assez de RAM : qwen2.5:1.5b (8 s par note, mais 87 % de notes avec une rubrique inventée).
    - qwen2.5:0.5b est écarté.
3. **À vérifier sur les 30 cas de validation (étape 4) :** l'avance de bge-m3 (15 cas, c'est peu), l'effet du garde-fou, et une version hybride TF-IDF + bge-m3.

## 7. Limites (à dire au jury)

- **15 cas seulement :** un cas représente 6,7 points de pourcentage. Le 100 % de bge-m3 est un très bon signe, pas une garantie : il faut le confirmer sur les 30 cas de validation.
- **Même auteur pour la banque et les cas de test** (données fictives) : risque de biais de formulation.
- **Le garde-fou et le plafond de 400 tokens ont été ajoutés après avoir vu des résultats.** Ce sont des règles générales, pas un réglage du prompt, mais il faut les confirmer sur les cas de validation.
- **Temps mesurés sous forte contrainte de RAM**, sur un seul PC et un seul passage → ce sont des ordres de grandeur, surtout pour le 3b.
- **Couverture :** `mxbai-embed-large` et `llama3.2:3b` ne sont pas testés.
- La recherche ne voit que le texte des questions : elle ne tient pas encore compte du score de réussite des questions (c'est le rôle du moteur V1).

## 8. Reproduire

```
# Si le disque C: est plein : une 2e instance d'Ollama avec les modèles sur D: (PowerShell)
$env:OLLAMA_MODELS='D:\ollama\models'; $env:OLLAMA_HOST='127.0.0.1:11435'; ollama serve
# Dans un autre terminal :
$env:OLLAMA_HOST='127.0.0.1:11435'
ollama pull nomic-embed-text; ollama pull all-minilm; ollama pull bge-m3
ollama pull qwen2.5:0.5b; ollama pull qwen2.5:1.5b; ollama pull qwen2.5:3b
cd data-brique
$env:OLLAMA_HOST='http://127.0.0.1:11435'
python benchmark/run_benchmark.py --pc pc_principal --embed nomic-embed-text all-minilm bge-m3 --llm qwen2.5:0.5b qwen2.5:1.5b qwen2.5:3b
python benchmark/exporter_pdf.py      # -> benchmark_llm.pdf

# Sur le PC d'un coéquipier (étiquette sans nom de personne), puis commiter benchmark/runs/pc2.json :
python benchmark/run_benchmark.py --pc pc2 --embed bge-m3 all-minilm --llm qwen2.5:3b
python benchmark/run_benchmark.py --rapport
```